CLI 명령어 처리 시스템 문서화

개요

IPFS Kubo의 CLI 명령어 처리 시스템은 사용자가 터미널에서 입력한 명령어를 파싱하고, 적절한 실행 환경을 선택하여 명령어를 실행하는 복잡한 시스템입니다. 이 시스템은 로컬 실행과 데몬을 통한 원격 실행을 모두 지원합니다.

CLI 아키텍처

전체 처리 흐름

```
사용자 입력 → 명령어 파싱 → 실행 환경 결정 → 명령어 실행 → 결과 출력
```

주요 컴포넌트

1. 진입점 (cmd/ipfs/main.go)

```go
func main() {
os.Exit(kubo.Start(kubo.BuildDefaultEnv))
}
```

모든 CLI 실행은 `kubo.Start()` 함수를 통해 시작됩니다.

2. 명령어 루트 (cmd/ipfs/kubo/ipfs.go)

```
var Root = &cmds.Command{
Options: commands.Root.Options,
Helptext: commands.Root.Helptext,
}


var localCommands = map[string]*cmds.Command{
"daemon": daemonCmd,
"init": initCmd,
"commands": commandsClientCmd,
}
```

CLI 전용 루트 명령어는 다음과 같은 특징을 가집니다:

- **로컬 전용 명령어**: `daemon`, `init`, `commands` 는 항상 로컬에서 실행
- **명령어 오버라이드**: 로컬 명령어가 원격 명령어를 오버라이드 가능
- **계층적 구조**: 하위 명령어들을 포함하는 트리 구조

명령어 파싱 과정

1. 인자 전처리

```
// Handle `ipfs --version'
if os.Args[1] == "--version" {
os.Args[1] = "version"
}
```

```
// Handle `ipfs help` and `ipfs help <sub-command>`
if os.Args[1] == "help" {
if len(os.Args) > 2 {
os.Args = append(os.Args[:1], os.Args[2:]...)
if os.Args[1] != "--help" {
os.Args = append(os.Args, "--help")
}
} else {
os.Args[1] = "--help"
}
}
```

명령어 파싱 전에 다음과 같은 전처리가 수행됩니다:

- `--version` → `version` 명령어로 변환
- `help <command>` → `<command> --help` 로 변환
- GUI 환경에서 인자 없이 실행 시 `daemon --init` 자동 실행

2. 명령어 해석

명령어는 다음과 같은 구조로 파싱됩니다:

```
ipfs <command> [<subcommand>] [<args>] [<options>]
```

예시:

- `ipfs add file.txt` → 명령어: `add`, 인자: `["file.txt"]`
- `ipfs dag get <cid>` → 명령어: `dag`, 하위명령어: `get`, 인자: `["<cid>"]`
- `ipfs cat <hash> --offset=100` → 명령어: `cat`, 인자: `["<hash>"]`, 옵션: `{offset: 100}`

실행 환경 결정

실행 모드 선택 로직

```
func makeExecutor(req *cmds.Request, env interface{}) (cmds.Executor, error) {
// 1. 명령어가 비활성화되었는지 확인
if req.Command.NoLocal && req.Command.NoRemote {
return nil, fmt.Errorf("command disabled: %v", req.Path)
}

// 2. 로컬 실행 가능 여부 확인
if !req.Command.NoLocal {
if doesNotUseRepo, ok := corecmds.GetDoesNotUseRepo(req.Command.Extra); doesNo
return exe, nil
}
}

// 3. API 주소 확인
apiAddr, err := apiAddrOption(req)
```

```
if err != nil {
return nil, err
}

// 4. 데몬 요청 여부 확인
daemonRequested := apiAddr != nil && req.Command != daemonCmd

// 5. 원격 실행 불가 명령어 처리
if req.Command.NoRemote {
if daemonRequested {
return nil, errors.New("api flag specified but command cannot be run on the dae
}
return exe, nil
}

// 6. API 주소 해결
if apiAddr == nil {
apiAddr, err = fsrepo.APIAddr(cctx.ConfigRoot)
// ...
}

// 7. 최종 실행 환경 결정
if apiAddr == nil {
```

```
if req.Command.NoLocal {
return nil, fmt.Errorf("command must be run on the daemon: %v", req.Path)
}
return exe, nil
}

// 8. HTTP 클라이언트 생성
return tracingWrappedExecutor{cmdhttp.NewClient(host, opts...)}, nil
}
```

실행 환경 유형

1. 로컬 실행 (Local Execution)

다음 조건에서 로컬 실행됩니다:

- 명령어가 `NoRemote` 플래그를 가지는 경우
- 데몬이 실행되지 않고 `NoLocal` 플래그가 없는 경우
- 저장소를 사용하지 않는 명령어 (`version` , `commands` 등)

```
// 저장소를 사용하지 않는 명령어 예시
var VersionCmd = &cmds.Command{
Extra: CreateCmdExtras(SetDoesNotUseRepo(true)),
// ...
}
```

2. 원격 실행 (Remote Execution)

다음 조건에서 데몬을 통해 원격 실행됩니다:

- `--api` 플래그로 API 주소가 지정된 경우
- 데몬이 실행 중이고 명령어가 `NoLocal` 플래그를 가지는 경우
- API 파일이 존재하는 경우

3. 폴백 실행 (Fallback Execution)

```
if !daemonRequested && fsrepo.IsInitialized(cctx.ConfigRoot) {
opts = append(opts, cmdhttp.ClientWithFallback(exe))
}
```

데몬 연결 실패 시 로컬 실행으로 폴백합니다.

명령어 컨텍스트 관리

Context 구조체

```
type Context struct {
ConfigRoot  string  // 설정 디렉토리 경로
ReqLog *ReqLog // 요청 로그
Plugins *loader.PluginLoader // 플러그인 로더
Gateway  bool  // 게이트웨이 모드 여부
api coreiface.CoreAPI // Core API 인스턴스
node *core.IpfsNode // IPFS 노드
```

```
ConstructNode func() (*core.IpfsNode, error) // 노드 생성 함수
}
```

지연 초기화 (Lazy Initialization)

```
func (c *Context) GetNode() (*core.IpfsNode, error) {
var err error
if c.node == nil {
if c.ConstructNode == nil {
return nil, errors.New("nil ConstructNode function")
}
c.node, err = c.ConstructNode()
}
return c.node, err
}
```

노드는 실제로 필요할 때까지 생성되지 않습니다. 이는 다음과 같은 이점을 제공합니다:

- **빠른 시작**: 노드가 필요하지 않은 명령어의 빠른 실행
- **메모리 절약**: 불필요한 리소스 사용 방지
- **에러 지연**: 노드 생성 에러를 실제 사용 시점까지 지연

API 인스턴스 관리

```
func (c *Context) GetAPI() (coreiface.CoreAPI, error) {
```

```
if c.api == nil {
n, err := c.GetNode()
if err != nil {
return nil, err
}
fetchBlocks := true
if c.Gateway {
cfg, err := c.GetConfig()
if err != nil {
return nil, err
}
fetchBlocks = !cfg.Gateway.NoFetch
}

c.api, err = coreapi.NewCoreAPI(n, options.Api.FetchBlocks(fetchBlocks))
if err != nil {
return nil, err
}
}
return c.api, err
}
```

옵션 및 인자 처리

옵션 파싱

명령어 옵션은 다음과 같이 정의됩니다:

```
var  CatCmd = &cmds.Command{
Arguments: []cmds.Argument{
cmds.StringArg("ipfs-path", true, true, "The path to the IPFS object(s)").Enable
},
Options: []cmds.Option{
cmds.Int64Option(offsetOptionName, "o", "Byte offset to begin reading from."),
cmds.Int64Option(lengthOptionName, "l", "Maximum number of bytes to read."),
cmds.BoolOption(progressOptionName, "p", "Stream progress data.").WithDefault(tr
},
// ...
}
```

인자 검증

```
Run: func(req *cmds.Request, res cmds.ResponseEmitter, env cmds.Environment) err
// 옵션 값 추출 및 검증
offset, _ := req.Options[offsetOptionName].(int64)
if  offset < 0 {
return  errors.New("cannot specify negative offset")
}
```

```
max, found := req.Options[lengthOptionName].(int64)
if max < 0 {
return errors.New("cannot specify negative length")
}
// 바디 인자 파싱
err = req.ParseBodyArgs()
if err != nil {
return err
}
// ...
}
```

전역 옵션

```
var Root = &cmds.Command{
Options: []cmds.Option{
cmds.StringOption(RepoDirOption, "Path to the repository directory to use."),
cmds.BoolOption(DebugOption, "D", "Operate in debug mode."),
cmds.BoolOption(OfflineOption, "Run the command offline."),
cmds.StringOption(ApiOption, "Use a specific API instance"),
// ...
},
```

```
}
```

전역 옵션은 모든 명령어에서 사용 가능합니다.

요청 로깅 시스템

ReqLog 구조

```
type ReqLogEntry struct {
StartTime time.Time // 시작 시간
EndTime time.Time // 종료 시간
Active  bool  // 활성 상태
Command  string  // 명령어 경로
Options  map[string]interface{} // 옵션들
Args []string  // 인자들
ID  int  // 고유 ID
}
```

로깅 과정

```
func (c *Context) LogRequest(req *cmds.Request) func() {
rle := &ReqLogEntry{
StartTime: time.Now(),
Active: true,
```

```
Command: strings.Join(req.Path, "/"),

Options: req.Options,

Args: req.Arguments,

log: c.ReqLog,

}

c.ReqLog.AddEntry(rle)



return func() {

c.ReqLog.Finish(rle)

}

}
```

요청 로깅은 다음과 같은 정보를 추적합니다:

- **실행 시간**: 명령어 시작/종료 시간
- **명령어 정보**: 실행된 명령어와 인자들
- **상태 추적**: 활성/완료 상태 관리
- **자동 정리**: 오래된 로그 엔트리 자동 제거

에러 처리 및 디버깅

디버그 모드

```
func checkDebug(req *cmds.Request) {

debug, _ := req.Options["debug"].(bool)
```

```
ipfsLogLevel, _ := logging.Parse(os.Getenv("IPFS_LOGGING"))

goLogLevel, _ := logging.Parse(os.Getenv("GOLOG_LOG_LEVEL"))



if debug || goLogLevel == logging.LevelDebug || ipfsLogLevel == logging.LevelDe

u.Debug = true

logging.SetDebugLogging()

}

}
```

디버그 모드는 다음 방법으로 활성화됩니다:

- `--debug` 플래그
- `GOLOG_LOG_LEVEL=debug` 환경변수
- `IPFS_LOGGING=debug` 환경변수 (deprecated)

에러 전파

```
func printErr(err error) int {

fmt.Fprintf(os.Stderr, "Error: %s\n", err.Error())

return 1

}
```

CLI에서 발생한 에러는 표준 에러로 출력되고 적절한 종료 코드를 반환합니다.

성능 최적화

프로파일링 지원

```
func profileIfEnabled() (func(), error) {
if os.Getenv(EnvEnableProfiling) != "" {
stopProfilingFunc, err := startProfiling()
if err != nil {
return nil, err
}
return stopProfilingFunc, nil
}
return func() {}, nil
}
```

`IPFS_PROF` 환경변수를 통해 CPU 및 메모리 프로파일링을 활성화할 수 있습니다.

추적 (Tracing)

```
type tracingWrappedExecutor struct {
exec cmds.Executor
}

func (twe tracingWrappedExecutor) Execute(req *cmds.Request, re cmds.ResponseEmi
ctx, span := tracer.Start(req.Context, "cmds."+strings.Join(req.Path, "."))
defer span.End()
```

```
err := twe.exec.Execute(req, re, env)

if err != nil {

span.SetStatus(codes.Error, err.Error())

}

return err

}
```

OpenTelemetry를 통한 분산 추적을 지원합니다.

특수 명령어 처리

GUI 환경 감지

```
if insideGUI() {

// GUI 환경에서 인자 없이 실행 시 데몬 시작

os.Args = append(os.Args, "daemon", "--init")

}
```

버전 호환성 검사

```
func getRemoteVersion(exe cmds.Executor) (*semver.Version, error) {

// 원격 데몬의 버전을 확인하여 호환성 검사

// ...

}
```

클라이언트와 데몬 간의 버전 호환성을 자동으로 검사합니다.

이러한 복잡한 CLI 처리 시스템을 통해 IPFS Kubo는 사용자에게 일관되고 직관적인 명령어 인터페이스를 제공하면서도, 내부적으로는 로컬 실행과 원격 실행을 투명하게 처리할 수 있습니다.